Laboratorio 11 — Cineclub: cartelera y hooks propios

Repositorio: https://github.com/leonel-m4/Desarrollo-de-Aplicaciones-Empresariales-LAB10

Rama de trabajo: `lab11`.

Fecha de verificación: 13 de noviembre de 2026.

1. Punto de partida y alcance

La rama remota `origin/lab11` ya existía y se creó su rama local de seguimiento. La revisión inicial encontró una bandeja de aprobaciones, no la cartelera descrita por la guía. Por ello se reconstruyó la cartelera con autorización del autor, conservando la composición anterior en `src/ApprovalApp.jsx` y sus pruebas de regresión. No se afirma haber partido de una cartelera LAB10 que no estaba presente.

La aplicación principal es ahora Cineclub, una cartelera de ocho películas con favoritos y una sección de cuatro estrenos ficticios. Los datos, fechas y valoraciones son de demostración; no representan disponibilidad actual en salas. Los pósteres ilustrados se incluyen localmente en SVG y no dependen de una API de imágenes.

2. Lista de requisitos y evidencia en código

Requisito	Implementación
Rama de sesión	`lab11`, con seguimiento de `origin/lab11`
Tipografía y utilidades	`src/cinema.css`: Outfit, DM Sans, alternativas del sistema, `.sr-only`, `.eyebrow`, botones, estados y ajustes adaptables
Patrón adaptador	`src/lib/movieAdapter.js`: `adaptMovie`; tanto cartelera como estrenos pasan por esta frontera
Aviso al añadir y quitar favorita	`MovieCard` comunica el evento; `App` cambia favoritas y mensaje; `Toast` presenta y cierra el aviso
Hook useDebounce	Temporizador de 300 ms en el buscador, con cancelación al cambiar valor o retardo y al desmontar
Hook useWindowSize	Ancho y alto actualizados con `resize`; el ancho decide las columnas
Hook usePremieres	Simulación de 900 ms con `loading`, `ready`, `error` y `reload`
Esqueleto y carrusel	`MovieSkeleton`, `PremiereCarousel` y `PremieresSection` representan los tres estados
Contador elevado	`App` deriva `favorites.length`; `MovieHeader` recibe solo `favoriteCount`, no la colección
Limpiar desde el banner	`MovieBanner` llama `onClearFilters`; `App` limpia búsqueda, género y filtro de favoritas
Árbol y justificación	Secciones 3 y 4 de este documento
Publicación y campus	Se comprueban por separado: publicar la rama no equivale a subir el PDF al campus

3. Árbol de componentes y datos de cada nivel

```
StrictMode
└── App
    Datos: movies, colección adaptada e inmutable a nivel de módulo
    Estado: favorites (IDs), query, genre, onlyFavorites, notification
    Hooks: useDebounce(query, 300), useWindowSize()
    Derivados: favoriteCount = favorites.length, visible, columns, hasFilters
    Referencia: sequence, identidad de cada aviso
    Efecto: persistir favorites en localStorage
    │
    ├── MovieHeader
    │   Props: favoriteCount, onlyFavorites, onShowFavorites
    │   Sin estado de favoritas ni acceso a la lista de películas
    │
    ├── main
    │   ├── MovieBanner
    │   │   Props: hasFilters, onClearFilters; sin copia local del filtro
    │   ├── section “En cartelera”
    │   │   Buscador controlado: query y setQuery de App
    │   │   Géneros: genre y setGenre de App
    │   │   Listado: visible, calculado por App
    │   │   └── MovieCard × N
    │   │       Props: movie, isFavorite, onToggleFavorite
    │   │       Sin estado duplicado ni conocimiento de Toast
    │   └── PremieresSection
    │       Estado local: failing, para simular error de conexión
    │       Hook: usePremieres -> status, premieres, error, reload
    │       ├── loading -> MovieSkeleton
    │       ├── error -> aviso y botón Reintentar
    │       └── ready -> PremiereCarousel
    │           Props: premieres adaptadas
    │           Referencia local: track, contenedor desplazable
    │
    ├── footer (contenido estático)
    └── Toast
        Props: notification, onDismiss
        Efecto local: cerrar a los 3200 ms y limpiar el temporizador
```

Los hooks mantienen sus propios detalles internos: `useDebounce` recuerda el valor estabilizado; `useWindowSize` recuerda las dimensiones; `usePremieres` recuerda la respuesta y el contador de recargas. El ancho no determina las películas filtradas, solo su distribución.

4. Por qué el contador de favoritas no vive en la cabecera

La lista de favoritas es necesaria para las tarjetas, el filtro de favoritas y el contador. La cabecera es hermana del contenido principal: no es ancestro de las tarjetas. Si fuera propietaria de la lista o de un contador independiente, los clics de las tarjetas no podrían actualizar todo de forma directa y se duplicaría información que habría que sincronizar.

El ancestro común más bajo que coordina estos consumidores es `App`. Allí vive la única fuente de verdad: `favorites`. El contador **no es otro useState**; se deriva como `favorites.length` en cada render. La cabecera recibe únicamente `favoriteCount` y se dedica a mostrarlo. Filtrar u ocultar tarjetas no cambia el total de favoritas.

El mismo criterio explica por qué el filtro vive en `App`: el buscador y el botón de limpieza del banner necesitan modificar el mismo dato. `MovieBanner` no toca el DOM del buscador ni mantiene una segunda búsqueda.

5. Patrón adaptador y flujo de eventos

Los datos externos tienen campos como `release_date`, `runtime`, `vote_average` y `poster_path`. `adaptMovie` los transforma a `year`, `duration`, `rating` y `poster`; además normaliza el ID como texto, limita la valoración y proporciona valores por defecto.

```
rawMovies -> map(adaptMovie) -> App -> MovieCard
rawPremieres -> map(adaptMovie) -> usePremieres -> PremiereCarousel

Clic en corazón
-> MovieCard llama onToggleFavorite(movie)
-> App actualiza favorites y notification
-> MovieHeader recibe el nuevo número
-> Toast presenta el mensaje y gestiona su duración
```

La tarjeta no importa `Toast`, no llama `alert` y no controla temporizadores de avisos. Se puede cambiar el tipo de notificación sin modificar `MovieCard`.

6. Hooks, estados y limpieza

useDebounce: el input muestra lo escrito inmediatamente; el listado utiliza el último valor que lleva 300 ms sin cambiar. Cada efecto cancela el temporizador anterior con `clearTimeout`. Al limpiar el banner, el campo se vacía inmediatamente y el listado se restablece al finalizar esos mismos 300 ms.

useWindowSize: lee las dimensiones iniciales y escucha `resize`; al desmontar elimina la misma función registrada. `App` usa cuatro tramos: menos de 560 px = 1 columna; 560–799 = 2; 800–1119 = 3; desde 1120 = 4.

usePremieres: inicia en `loading`, sin datos ni error. Tras 900 ms entrega datos adaptados y `ready`, o un error y `error` si se activó el fallo. `reload` inicia otra carga y limpia el error anterior. La limpieza cancela la petición simulada previa, evitando que una respuesta antigua sobrescriba una nueva o actualice después de desmontar. Funciona con StrictMode.

Toast: cada mensaje tiene una identidad nueva. El temporizador se reinicia al cambiar de aviso y se cancela al desmontar. También puede cerrarse con su botón.

7. Comprobaciones automatizadas

Comandos ejecutados en PowerShell:

```
npm.cmd ci
npm.cmd test
npm.cmd run build
```

Resultado final de las pruebas: **75 pasan, 0 fallan, 8 archivos aprobados**.

Archivo	Pruebas
`Cinema.test.jsx`	12: adaptador, evento desacoplado, cabecera, favoritas, avisos, persistencia, búsqueda, limpieza, géneros, columnas y error
`usePremieres.test.jsx`	4: estados, recarga, cancelación y desmontaje con StrictMode
`useWindowSize.test.jsx`	2: redimensionado y limpieza del listener
`useDebounce.test.jsx`	10: retardo, reinicio y limpieza
`usePlans.test.jsx`	12 de regresión de la bandeja preservada
`useAgentRun.test.jsx`	18 de regresión de la bandeja preservada
`App.test.jsx`	7 sobre la composición anterior, ahora `ApprovalApp`
`Inbox.test.jsx`	10 de regresión

La prueba anterior de App cambió únicamente su ruta de importación a `ApprovalApp`; no se alteraron expectativas para fingir que una cartelera es una bandeja. Las pruebas nuevas importan la cartelera real desde `src/App.jsx`.

La compilación de producción genera `dist/`. Las evidencias de consola se guardan en `docs/evidencias-lab11/`.

Dependencias: la instalación reportó tres vulnerabilidades (una moderada y dos críticas). No se aplicaron actualizaciones mayores automáticas; una compilación correcta no equivale a una auditoría de seguridad.

8. Comprobación reproducible de la interfaz

1. Abrir `http://localhost:5173`: comprobar ocho tarjetas y la carga de estrenos.

2. Pulsar el corazón de Interestelar: comprobar aviso de añadido y contador en uno; pulsar otra vez: aviso de eliminado y contador en cero.

3. Escribir “Dune”: antes de 300 ms permanece la colección anterior; después solo aparece Dune.

4. Pulsar “Limpiar filtros” en el banner: el campo queda vacío y, tras el debounce, regresan las ocho tarjetas.

5. Combinar “Mis favoritas” con un género: el contador global no debe cambiar.

6. Redimensionar a 390, 700, 1000 y 1280 px: comprobar una, dos, tres y cuatro columnas.

7. Pulsar “Recargar estrenos”: aparece el esqueleto y después el carrusel.

8. Marcar “Simular error de conexión”: aparece el aviso y “Reintentar”. Desmarcarlo permite recuperar los estrenos.

9. Usar las flechas del carrusel o desplazarlo horizontalmente.

10. Recargar la página: las favoritas se recuperan desde `cineclub-favorites`.

Las pruebas automatizadas verifican estos contratos funcionales. La comprobación en navegador y las capturas se documentan por separado en las evidencias; no se atribuyen a una revisión manual del alumno.

9. Autoría, publicación y presentación

Implementación y documentación preparadas con asistencia de IA a solicitud del autor; el equipo debe revisar el resultado antes de presentarlo. Los entregables incluidos en esta rama corresponden únicamente al laboratorio 11.

La rama de entrega es `lab11`. El envío del PDF al campus requiere la cuenta del alumno y una confirmación de recepción del propio campus: **no está realizado por el asistente ni se considera demostrado por la presencia del archivo en GitHub**.